CC3092 Deep Learning y Sistemas Inteligentes
Proyecto 3: Casos de Uso de LLM

1. Descripción general

El proyecto consiste en diseñar y construir un **producto end-to-end** basado en uno o varios Large Language Models (LLMs) de elección del grupo. El producto debe resolver un problema real para un usuario concreto y estar listo para que cualquier persona lo pruebe. **No tiene que ser una interfaz de chat necesariamente:** cualquier caso de uso que un LLM pueda resolver es válido (clasificar, extraer información estructurada, resumir, traducir, recomendar, generar contenido, automatizar tareas, buscar por significado, entre otros).

El objetivo central no es únicamente que el producto "funcione", sino que el grupo **entienda y documente todo el pipeline de datos**, desde la fuente de información hasta la respuesta que ve el usuario, y que **mida de forma objetiva la calidad de las respuestas** obtenidas. Se calificará la **creatividad**, la **utilidad** y la **utilizabilidad** del producto final.

La presentación será estilo **feria científica**: cada grupo tendrá un pequeño stand alrededor de la clase donde mostrará su producto funcionando en vivo. Posteriormente, el grupo entregará un trabajo escrito con toda la descripción técnica del proyecto.

Información clave

Modalidad	Grupos de 1, 2 o 3 personas
Presentación (presencial)	Lunes 2 de noviembre de 2026, durante el periodo de clase. Formato feria científica.
Entrega del trabajo escrito	Viernes 6 de noviembre de 2026
Modelo a utilizar	LLM(s) a elección: cerrados vía API (GPT, Claude, Gemini, entre otros) o abiertos (Llama, Qwen, DeepSeek, Mistral, Gemma, entre otros)
Técnicas permitidas	Prompting (zero/few-shot), RAG con embeddings, uso de herramientas/agentes, fine-tuning (completo o eficiente, p. ej. LoRA), o una combinación
Producto	Aplicación funcional end-to-end con interfaz usable (web, móvil, bot, extensión, plugin, CLI, etc.)
Criterios de la feria	Creatividad, utilidad, utilizabilidad y dominio técnico
Entregables adicionales	Repositorio de GitHub y video corto de demostración del producto (1 a 3 minutos, .mp4)

Rúbrica

Componente	**Puntos**	**Total**
Trabajo escrito	4 pts	4
Feria – Creatividad	1 pt	1
Feria – Utilidad	1 pt	1
Feria – Utilizabilidad	1 pt	1
Feria – Dominio técnico (pipeline y medición de resultados)	1 pt	1
Total		8

Requisitos mínimos del producto

- **End-to-end:** debe cubrir el flujo completo: entrada del usuario o de datos → procesamiento → LLM → salida que el usuario puede usar directamente. No se aceptan notebooks sueltos como producto final.
- **Utilizable por terceros:** una persona que no forma parte del grupo debe poder usar el producto.
- **Medible:** el grupo debe contar con un conjunto de evaluación propio y métricas que permitan cuantificar la calidad de las respuestas (ver sección 2.4).
- **Justificado:** cada decisión técnica (modelo, técnica de adaptación, preprocesamiento, prompts) debe estar explicada y respaldada por resultados.

Ejemplos de casos de uso (solo como inspiración)

- Extracción estructurada: convertir facturas, recibos, CVs o actas en datos tabulares listos para una hoja de cálculo o base de datos.
- Búsqueda semántica y RAG sobre un corpus específico: reglamentos universitarios, documentación técnica, jurisprudencia o manuales de un curso.
- Clasificación y enrutamiento: priorizar correos, tickets de soporte o reseñas por intención, urgencia o sentimiento.
- Educación: generador de ejercicios y retroalimentación automática a partir del material de una clase.
- Programación: revisor de pull requests, generador de pruebas unitarias o explicador de código heredado.
- Lenguaje y cultura local: apoyo a la traducción o al aprendizaje de idiomas con pocos datos digitales (por ejemplo, idiomas mayas), documentando sus limitaciones.
- Agentes: un sistema que usa herramientas (APIs, calendario, hojas de cálculo, búsqueda web) para completar una tarea de varios pasos.

2. Estructura del trabajo escrito

2.1 Definición del problema y usuario objetivo

Presenta el problema que resuelve el producto y por qué un LLM es una herramienta adecuada para resolverlo.

- ¿Quién es el usuario objetivo y qué tarea concreta le resuelve el producto? ¿Cómo la resolvía antes?
- ¿Por qué este problema requiere un LLM? ¿Se consideraron alternativas más simples? Justifica la elección.
- Criterios de éxito: ¿qué significa que el producto funcione “bien”? Define desde aquí qué se va a medir.

2.2 Datos y pipeline del sistema

Describe el recorrido completo de la información, análogo al EDA y al preprocesamiento de un problema supervisado.

- Fuentes de datos utilizadas (documentos, APIs, bases de datos, entradas del usuario), su volumen, formato y calidad.
- Preprocesamiento: limpieza, normalización, segmentación en fragmentos (chunking) y su tamaño, manejo de metadatos.
- Si se utiliza RAG o búsqueda semántica: modelo de embeddings, dimensión de los vectores, métrica de similitud (p. ej. similitud coseno), base vectorial y número de fragmentos recuperados (top-k).
- **Diagrama de arquitectura** del pipeline completo: desde la entrada hasta la salida, incluyendo cada componente (tokenización, embeddings, recuperación, LLM, post-procesamiento, interfaz).
- Consideraciones de la ventana de contexto: cuántos tokens consume una solicitud típica y cómo se controla.

2.3 Selección del modelo y técnica de adaptación

- LLM(s) evaluados y justificación de la elección final: abierto vs. cerrado, base vs. instruct, tamaño (parámetros), denso vs. MoE, costo por solicitud, latencia y licencia.
- Técnica de adaptación utilizada (prompting, few-shot, RAG, fine-tuning, uso de herramientas) y su relación con los conceptos vistos en clase (transfer learning a escala).
- Diseño de los prompts finales: instrucciones de sistema, ejemplos incluidos, formato de salida esperado (p. ej. JSON) y parámetros de generación (`temperature`, `top_p`, `max_tokens`).
- Si hubo fine-tuning: datos de entrenamiento, hiperparámetros, recursos de cómputo y comparación contra el modelo sin ajustar.

2.4 Evaluación y medición de respuestas

Un LLM predice texto probable, no necesariamente texto verdadero: el grupo debe demostrar con evidencia qué tan bien responde su sistema.

- **Conjunto de evaluación propio** (se recomienda un mínimo de 30 a 50 casos) con entradas representativas y la respuesta o resultado esperado. Describe cómo se construyó y quién validó las respuestas de referencia.
- Métricas adecuadas al caso de uso, por ejemplo: exactitud, precisión, recall y F1 (clasificación o extracción); recall@k o MRR (recuperación); fidelidad al contexto y tasa de alucinaciones (RAG); evaluación humana con una rúbrica; LLM-as-a-judge (validado contra evaluación humana en una muestra).
- Métricas operativas: latencia promedio por solicitud, tokens consumidos y costo estimado por uso.
- Comparación contra al menos un **baseline** (p. ej. el LLM sin RAG, un prompt simple, un modelo más pequeño o un método sin LLM).

2.5 Resultados de iteraciones

Documenta el historial de experimentos. Se recomienda una tabla que resuma cada iteración:

- Identificador de la iteración y cambios realizados respecto a la anterior (modelo, prompt, tamaño de chunk, top-k, modelo de embeddings, parámetros de generación).
- Valor de las métricas definidas en la sección 2.4 para cada iteración.

- Ejemplos concretos de fallas (alucinaciones, formato incorrecto, respuestas fuera de tema, recuperación de contexto irrelevante) y cómo se abordaron.

2.6 Producto final y experiencia de usuario

- Descripción del producto final con capturas de pantalla y el flujo de uso principal.
- Decisiones de diseño orientadas a la utilizabilidad: manejo de errores, tiempos de espera, cómo se le comunica al usuario la incertidumbre del modelo.

2.7 Limitaciones, riesgos y consideraciones éticas

- Limitaciones técnicas observadas: alucinaciones, fecha de corte del modelo, ventana de contexto, costo y latencia.
- Riesgos del producto: sesgos, privacidad de los datos del usuario, prompt injection y uso indebido. ¿Qué medidas se tomaron?
- ¿En qué situaciones **no** debería usarse el producto?

2.8 Conclusiones

- Desempeño final del sistema según las métricas definidas y su interpretación en el contexto del problema.
- Principales aprendizajes técnicos y metodológicos del proceso.
- Posibles mejoras o líneas de trabajo futuro si se contara con más tiempo, datos o presupuesto.

2.9 Enlace al repositorio de GitHub

Incluye el enlace al repositorio público (o con acceso otorgado al curso) que contenga el código completo del proyecto: ingesta y preprocesamiento de datos, construcción del índice (si aplica), prompts, aplicación, conjunto de evaluación y scripts de evaluación. El README debe explicar cómo instalar y ejecutar el producto y cómo reproducir las métricas reportadas. **Nunca** se deben subir llaves de API al repositorio: usar variables de entorno (por ejemplo, un archivo `.env` excluido en `.gitignore`).

3. Feria científica

El lunes 2 de noviembre de 2026, durante el periodo de clase, cada grupo montará un pequeño stand donde presentará su producto a quienes lo visiten. La dinámica será la siguiente:

1. Cada grupo debe tener su producto funcionando en vivo en al menos un dispositivo, listo para que los visitantes lo usen directamente.
2. En cada visita, el grupo realizará un pitch breve (3 a 5 minutos) que cubra: el problema y el usuario objetivo, una demostración del producto, el diagrama del pipeline y las métricas de evaluación obtenidas.
3. El stand debe contar con material de apoyo visual (póster impreso o presentación en pantalla) que incluya el diagrama de arquitectura y los resultados principales.
4. Durante la visita del catedrático, cualquier integrante del grupo debe poder responder preguntas técnicas sobre cualquier parte del pipeline. La evaluación se realizará con los criterios de la tabla siguiente.

Criterios de evaluación de la feria

Criterio	¿Qué se evalúa?
Creatividad	Originalidad del caso de uso y de la solución. Se valora ir más allá de un chat genérico de preguntas y respuestas.
Utilidad	Relevancia del problema resuelto para un usuario real y valor que el producto aporta frente a las alternativas existentes.
Utilizabilidad	Facilidad de uso, claridad de la interfaz, tiempos de respuesta y robustez ante entradas inesperadas.
Dominio técnico	Comprensión del pipeline de datos de punta a punta y capacidad de explicar cómo se midieron y cómo se interpretan las respuestas del modelo.

Recomendaciones

1. Definir el caso de uso y el usuario objetivo cuanto antes, y validar la idea con potenciales usuarios antes de construir.
2. Construir el conjunto de evaluación al inicio del proyecto, no al final: permite comparar objetivamente cada cambio de modelo o de prompt.
3. Versionar los prompts y registrar cada iteración (por ejemplo, con una hoja de cálculo, MLflow o Weights & Biases) para facilitar la redacción de la sección de resultados.
4. Preparar un plan de respaldo para el día de la feria: la conexión a internet o una API externa pueden fallar. Tener el video de demostración listo y, si es posible, respuestas de ejemplo en caché.
5. Estimar y controlar el costo de las APIs desde el inicio (límites de gasto, modelos más pequeños para desarrollo) y probar el producto con la carga esperada durante la feria.
6. Reutilizar los conceptos vistos en clase (tokenización, embeddings, similitud coseno, RAG, transfer learning) y explicar en qué parte del pipeline se aplica cada uno.
7. Priorizar un producto pequeño que funcione muy bien sobre uno ambicioso que funcione a medias.